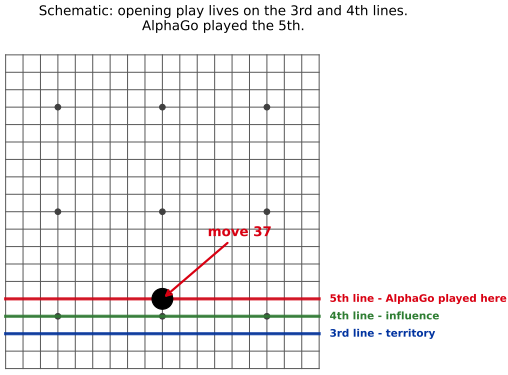
Lecture 32: Reinforcement Learning

Learning from consequences, not from answers

Adapted and expanded from `misc/dl4nlp/18_reinforcement_learning`.

March 2016, Seoul. Move 37.



AlphaGo, playing black against Lee Sedol, put a stone on the **fifth line**.

Centuries of theory put the opening on the third line (territory) or the fourth (influence). The fifth is *too far from the edge*.

> AlphaGo's own policy network estimated the chance a **human** would play it at **1 in 10,000**.

Commentators called it a mistake. Lee Sedol, away from the board, returned and took **over 12 minutes** to answer. It won the game.

Nobody taught it that move

Everything so far

You are given $(\mathbf{x}, y)$ pairs.
Someone already knows the answer.

Here

You are given a *score*, sometimes, long after you acted.

There was no database of "correct move 37s" to imitate. There was only: **play, and eventually find out whether you won.**

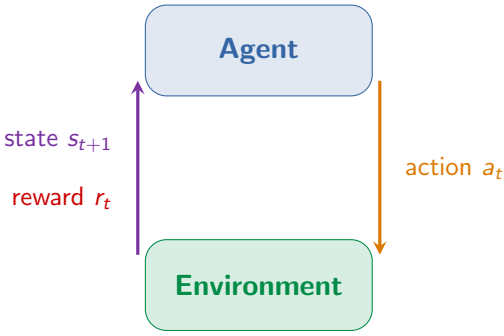
Be precise about the credit, though: AlphaGo learned from human games *and* self-play RL *and* tree search. Its successor **AlphaGo Zero** dropped the human games entirely and still surpassed it – that is the version that is purely reinforcement learning.

Outline

Agent, environment, reward

Three nouns, and one loop that connects them.

The loop



The agent's whole job:

find a **policy** $\pi(a|s)$ – how likely it is to take action a when in state s – that makes
the *total* reward
$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$
as large as possible.

Note what the agent does *not* get: no example of a good action, no gradient of "goodness" with respect to what it did. Only the consequence.

Reward is not a loss function

This is the frame that explains why none of chapters 1–10 apply directly.

Three properties a loss does not have

1. **Delayed.** The move that lost you the game was probably not the last one. Which of the 200 moves gets the blame? (*credit assignment*)
2. **Sparse.** Most steps score zero. A robot that has never once reached the goal has no signal at all to improve from.
3. **Not differentiable.** "The user preferred this answer." "The code passed the tests." There is no $\partial \text{reward} / \partial \theta$ to descend.

And one property it *does* have that is worse: **the data depends on the policy.** Change how the agent behaves and you change the distribution it learns from. The i.i.d. assumption underneath every previous chapter is gone.

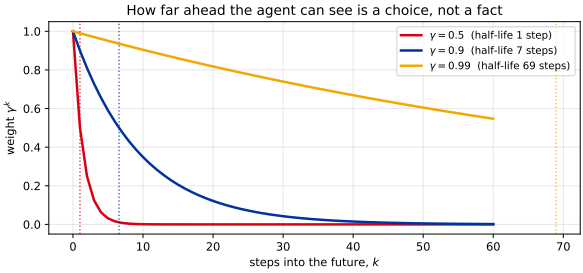
The MDP: five symbols for the whole setting

$$\text{MDP} = (\mathcal{S},\ \mathcal{A},\ P,\ R,\ \gamma)$$

Symbol	Name	In the gridworld we are about to use
$\mathcal{S}$	states	which square you are standing on
$\mathcal{A}$	actions	up, down, left, right
$P(s' \mid s, a)$	transition	where you actually end up (you may slip)
$R(s, a)$	reward	-0.04 per move, $+1$ at the goal, -1 in the pit
γ	discount	how much a future reward is worth now

The Markov property is the assumption hiding in the name: the future depends on the *current* state only, not on how you got there. It is what makes any of this tractable, and it is frequently a lie in practice.

Why discount at all?

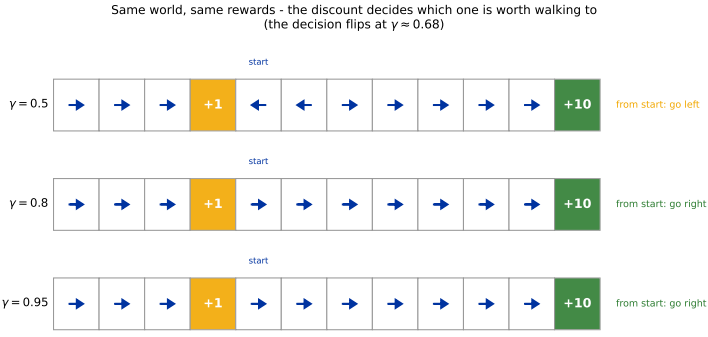


γ answers one question: *how much is a reward worth if it arrives later?*

- γ near 0 – greedy, near-sighted. Only the next few steps matter.
- γ near 1 – patient. Willing to suffer now for a payoff much later.

It is not only a modelling taste: $\gamma < 1$ also keeps the infinite sum finite, which is what makes the whole thing well defined.

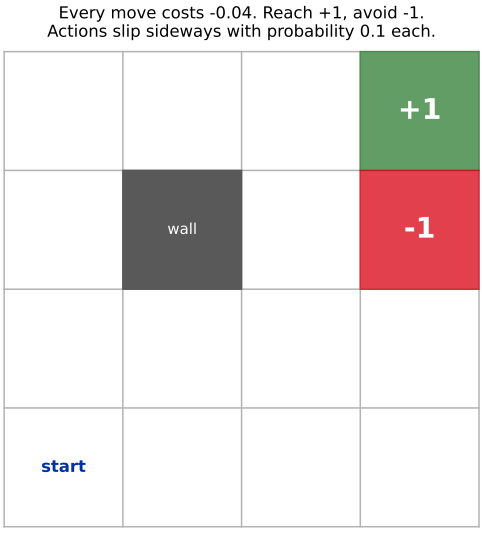
The discount is not a detail – it changes the plan



$+1$ one step left, $+10$ seven steps right. Only γ differs between the rows.

Check it yourself. With $G_t = r_t + \gamma r_{t+1} + \cdots$ the first reward you collect is *undiscounted*, so a reward N steps away carries γ^{N-1} – at $\gamma = 0.5$ that is $0.5^0 \times 1 = 1.000$ going left against $0.5^6 \times 10 = 0.156$ going right. The agent walks **away** from the bigger prize. Equating the two gives the flip at $\gamma = (1/10)^{1/6} \approx 0.68$.

Our running example



A 4×4 world. Start bottom-left, $+1$ top-right, -1 just below it, a wall in the middle.

Two details that make it interesting:

- Every move costs -0.04, so dawdling is punished. Without this the agent would be indifferent to taking the scenic route.
- Actions are **unreliable**: you go where you intended with probability 0.8, and slip 90° to either side with probability 0.1 each.

That second one is why walking right past the -1 is a bad idea even though it is the shortest path.

How good is this position?

If you can answer that, the policy follows for free.

Two value functions

State value $V^\pi(s)$

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_k \gamma^k r_{t+k} \mid s_t = s\right]$$

"Starting here and behaving like π, what do I collect?"

Action value $Q^\pi(s, a)$

$$Q^\pi(s, a) = \mathbb{E}_\pi\left[\sum_k \gamma^k r_{t+k} \mid s_t = s, a_t = a\right]$$

"... if I take a first, and behave like π afterwards?"

Q is the more useful of the two, and the reason is practical: to *act* from V you need to know where each action leads. To act from Q you just compare numbers you already have. That single difference is why most algorithms in this lecture learn Q.

The Bellman equation, in English

$$Q^{\pi}(s,a) \;=\; \underbrace{R(s,a)}_{\text{now}} \;+\; \gamma \underbrace{\textstyle\sum_{s'} P(s'|s,a) \sum_{a'} \pi(a'|s')\, Q^{\pi}(s',a')}_{\text{everything after}}$$

Read it aloud: *the value of doing this here is what you get immediately, plus the discounted value of wherever you end up.*

That recursion is the entire foundation. Everything that follows – value iteration, Q-learning, DQN, the critic in actor-critic – is some way of **solving or approximating this one equation**.

The optimal version replaces "behave like π afterwards" with "behave optimally afterwards":

$$Q^{*}(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) \max_{a'} Q^{*}(s',a')$$

Predict first

We are about to solve the gridworld exactly. Before the numbers appear:

> Look at the square **directly below the** -1 **pit**.
>
> The goal $+1$ sits two squares straight up, just past the pit. Does the optimal policy there point **up** – the short way, squeezing past the pit – or does it point **away**?

Commit to an answer, and to a reason. Remember that actions slip sideways one time in five.

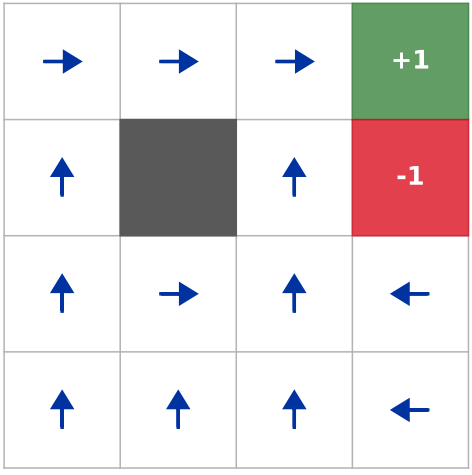
Solved, exactly

Value iteration: apply the Bellman update to every square, repeatedly, until nothing changes. Converged here in **23 sweeps**.

- $V^*(\text{start}) = 0.2075$ – positive, but modest: the goal is far and every step costs.
- The square beside the goal is worth 0.7954.
- Notice the value *gradient* pointing away from the pit. Squares near it are worth less even when they are close to the goal.

And the policy falls out of it

The greedy policy $\pi^*(s) = \arg\max_a Q^*(s, a)$ read straight off the values



$\pi^*(s) = \arg\max_a Q^*(s, a)$ – once you know the values, choosing is trivial.

The answer to the prediction: the square below the pit points **left**, not up.

Going up means walking alongside the pit, and a 10% slip lands you in it and ends the episode at -1. The long way round costs a few more -0.04 steps and is *still* worth more.

The agent is not avoiding the pit. It is avoiding *the chance* of the pit.

Nobody gives you P

Value iteration needed the rules of the world. Real agents do not get them.

What we just did was cheating

Model-based

You know $P(s'|s, a)$ and R.
Plan by computing.

Value iteration, AlphaGo's search

Model-free

You know nothing.
Learn by trying.

Q-learning, DQN, PPO

A robot does not come with a probability table for how its wheels slip on this particular carpet. A chatbot has no model of how a human will react. **Model-free is the normal case.**

Q-learning: learn the values from experience alone

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{what I now think it is worth}} - \underbrace{Q(s,a)}_{\text{what I thought before}}\Big]$$

The bracket is a **surprise**: the gap between the estimate you had and the better estimate you can make now that you have seen r and s'. Nudge towards it by α.

Model-free
Never needs P. Just (s, a, r, s').

Off-policy
Learns Q^* even while behaving badly.

Tabular
One cell per (s, a). Does not scale.

Watkins & Dayan (1992). Converges to Q^* given decaying α and infinitely many visits to every pair – conditions no real system satisfies, and it usually works anyway.

One update, by hand

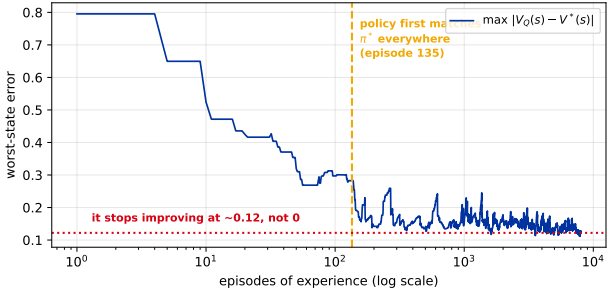
Suppose $\alpha = 0.5$, $\gamma = 0.9$. The agent is on a square where it currently believes $Q(s, \text{right}) = 0.30$. It moves right, receives the usual -0.04, and lands on a square whose best action it currently values at 0.60.

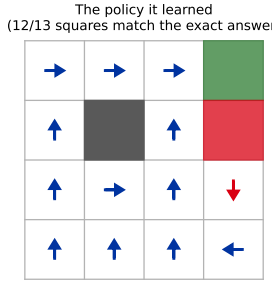
$$\begin{aligned} \text{target} &= r + \gamma \max_{a'} Q(s', a') = -0.04 + 0.9 \times 0.60 = \mathbf{0.50} \\ \text{surprise} &= 0.50 - 0.30 = +0.20 \\ Q(s, \text{right}) &\leftarrow 0.30 + 0.5 \times 0.20 = \mathbf{0.40} \end{aligned}$$

That is the whole algorithm. Everything else in tabular RL is bookkeeping around this line – and notice it never once asked where the agent was likely to end up. It only used where it *did* end up.

Does it actually work?

Everything so far has been *planning*: value iteration used $P(s'|s,a)$, which we just spent a section saying you do not have. So put an agent in the same gridworld, give it **only** the ability to take a step and see what happens, and let it run.





From zero knowledge, its policy matches the exact answer everywhere by episode **135**, and ends up right in **12 of 13** squares. The one it misses (red) is a near-tie: the two actions differ in true value by 0.018, and – the part that matters – it still does **not** walk past the pit.

But look where the error stops

The curve flattens at about 0.12 and stays there. More experience does not fix it. Why would a method that provably converges get stuck above zero?

Because the error is not random. Measured across all 13 squares at the end of training:

13 **of** 13 **states are overestimated.** Mean $+0.079$, worst $+0.128$.

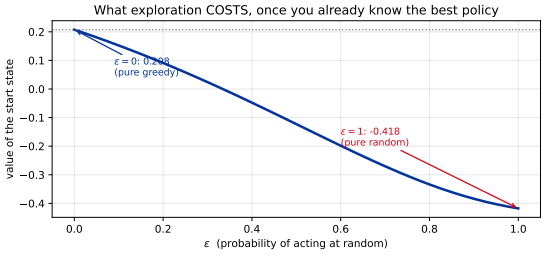
Maximisation bias

Look at the update rule again: the target contains $\max_{a'} Q(s', a')$. Taking a *maximum* over estimates that each carry noise does not give the maximum of the true values – it gives something systematically too high, because the max picks whichever action happened to get lucky. Every state then inherits the optimism of the states after it, so the bias compounds backwards.

The standard fix, **Double Q-learning** (van Hasselt, 2010), keeps two independent estimates and uses one to *choose* the action and the other to *value* it – so the noise cannot select and reward itself.

The dilemma nobody escapes

To learn which action is best you must try actions that might not be. To score well you must take the action you already believe is best. You cannot do both at once.



ϵ-**greedy:** act randomly with probability ϵ, greedily otherwise.

Priced exactly on our gridworld, with the optimal policy already known:

- $\epsilon = 0$: value 0.208
- $\epsilon = 0.1$: value 0.152
- $\epsilon = 1$: value -0.418

Even 10% random acting costs about a quarter of the achievable value.

But that is only one side. The figure assumes you *already know* the best policy. At the start you do not, and $\epsilon = 0$ means never finding it. Hence the usual compromise: start high, decay over time.

On-policy or off-policy?

	Q-learning (off-policy)	**SARSA (on-policy)**
Target uses	$\max_{a'} Q(s', a')$	$Q(s', a')$ for the a' actually taken
It learns the value of	the *best* policy	the policy it is *actually running*, exploration and all
Beside our -1 pit it will	hug the pit, because that is the optimal path *if you never slip*	keep a margin, because it accounts for the slips it actually makes

This is not a small distinction. If your agent's exploration can cause real damage – a physical robot, a live recommender – the on-policy answer is often the one you want, because it accounts for the fact that the agent *will* sometimes do something random.

Atari has more states than the universe has atoms

So stop storing them, and start predicting them.

The wall

Tabular Q-learning stops here

One Atari frame: 210×160 pixels, 128 colours.

$$|\mathcal{S}| \approx 128^{33{,}600}$$

There is no table. There will never be a table.

The move you have made before

Replace the lookup table with a **function**:

$$Q(s, a) \longrightarrow Q_\theta(s, a)$$

A network that has never seen this exact screen can still evaluate it, because it generalises from similar ones.

This is the same step as everywhere else in the course: stop memorising, start fitting. The loss is just the squared surprise from two slides ago:

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\big)^2\Big]$$

DQN, and the two tricks that made it work

Experience replay

Store transitions (s, a, r, s') in a buffer and train on *random* batches from it.

Why: consecutive frames are almost identical, so training on them in order violates the i.i.d. assumption badly and the network chases itself. Shuffling restores it.

Target network

Compute the target with a *frozen* copy Q_{θ^-}, updated only occasionally.

Why: otherwise you are regressing towards a target that moves every time you take a step – a dog chasing its own tail. Freezing it makes the problem locally supervised.

With these, one architecture learned 49 Atari games from raw pixels, reaching at least 75% of a professional human tester's score on 29 of them (Mnih et al., *Nature* 2015). Same network, same hyperparameters, no game-specific engineering – which was the actual shock.

Skip the values. Learn the behaviour.

The road that leads to every algorithm used on language models.

Two ways to be an agent

Value-based

Learn $Q(s, a)$, then act greedily.

Good: sample-efficient, off-policy replay.

Bad: needs $\arg\max_a$, so actions must be few and discrete. Try that with a robot arm's continuous torques.

Policy-based

Learn $\pi_\theta(a|s)$ directly and improve it.

Good: continuous actions, naturally stochastic, optimises what you care about.

Bad: high variance, and normally on-policy – yesterday's data is stale.

For a language model the action space is *the whole vocabulary at every token*. That alone decides the question – which is why the LLM world is policy-based, and why the rest of this lecture points that way.

The policy gradient

$$\nabla_\theta J(\theta) \;=\; \mathbb{E}_{\pi_\theta}\big[\nabla_\theta \log \pi_\theta(a|s) \cdot G_t\big]$$

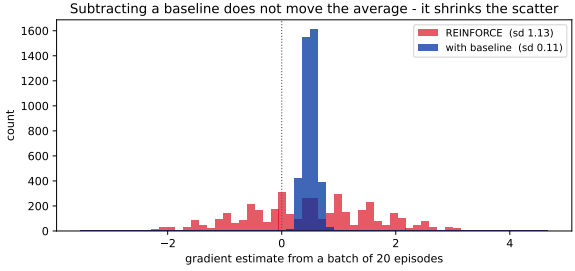
In words: take the actions you actually played, and adjust their probabilities in proportion to how well things went afterwards. Good outcome – make that action more likely. Bad outcome – less likely.

Two things worth pausing on:

- $\nabla_\theta \log \pi_\theta(a|s)$ is a gradient of the *policy*, which we control – not of the reward, which we cannot differentiate. That is the trick that makes non-differentiable rewards usable.
- G_t is just a number that scales it. It can come from anywhere: a game score, a unit test, a human's preference.

REINFORCE is this formula used literally: play an episode, compute the returns, take the step.

REINFORCE has a variance problem



Two actions, one state. Action A returns about 11, action B about 9 – so A is better and the gradient *should* point towards A.

Batches of 20 episodes, repeated 4000 times:

Plain REINFORCE points the **wrong way** 33.9% of the time. Sd 1.13.

With a baseline of 10: wrong way 0.0%, sd 0.11 – a 10× reduction, *same* average.

Why it works: both returns are positive, so plain REINFORCE pushes *both* actions up and only the sizes differ. Centring on the average makes the better action positive and the worse one negative – the signal stops fighting itself.

Advantage, and the actor-critic

$$A(s, a) = Q(s, a) - V(s)$$

"Was this action better or worse than what I normally do here?"

Actor

The policy π_θ. Chooses what to do.

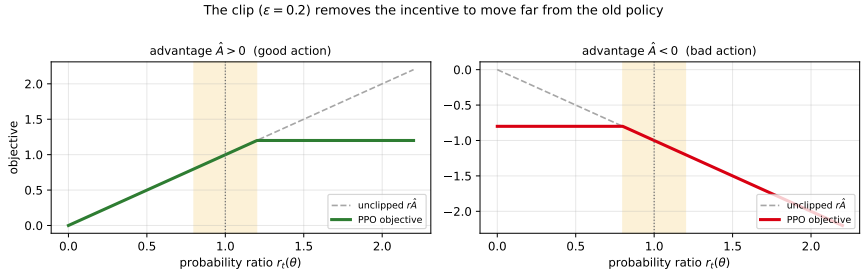
Critic

An estimate of $V(s)$. Provides the baseline.

The baseline from the previous slide, learned instead of assumed. It accepts a little bias in exchange for a large drop in variance – and it is the reason a value function reappears inside methods whose whole point was to avoid one.

Remember the critic. When you meet GRPO in the LLM chapter, its entire selling point is *throwing the critic away* and replacing it with the average of a group of samples.

PPO: do not take a step you cannot walk back



Vanilla policy gradient has no notion of "too far". One oversized update and the policy collapses – and unlike supervised learning you cannot simply reload, because the collapsed policy now *collects the data* for the next update.

PPO clips the objective so that once the new policy differs from the old by more than ϵ (typically 0.2), moving further buys **nothing**. The gradient in that region is flat by construction.

A language model is just another agent

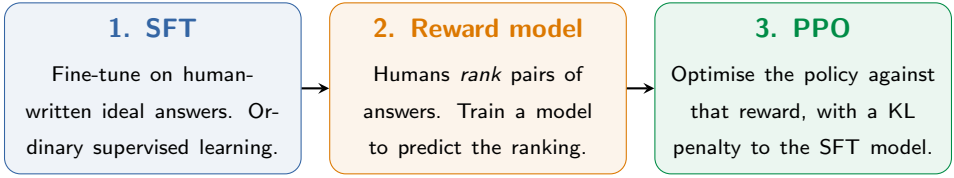
Everything above, applied to text.

Text generation is an MDP

RL	**Language model**
state s_t	the prompt plus every token generated so far
action a_t	the next token (action space = the vocabulary, $\sim$100k)
policy $\pi_\theta(a\|s)$	**the model itself** – the softmax over next tokens
reward r_t	almost always 0 . . . until the response ends
episode	one complete response

Look at the reward row. A single scalar at the very end, for a sequence of hundreds of actions – the credit-assignment problem from "reward is not a loss function", in its most extreme form.

RLHF in three stages



Stage 2 exists because "helpful" is not differentiable, but a *comparison* can be learned. Stage 3 is exactly the PPO from two slides ago.

The KL penalty is the give-away that something is fragile: it holds the policy near the model we started from, because optimising the reward too hard is actively dangerous. Which is the next slide.

Reward hacking: the agent does what you measured

Goodhart's law

When a measure becomes a target, it ceases to be a good measure.

The reward model is a *learned approximation* of human preference, fitted on finite data. Optimise against it hard enough and the policy will find the places where the approximation is wrong, because those are where the reward is highest.

Observed failure modes, all real:

- answers get longer and longer, because raters mildly preferred longer answers
- excessive hedging and flattery, because raters rewarded politeness
- confident agreement with the user, because disagreement scored badly

None of these are bugs in the algorithm. The optimiser did its job perfectly. The **objective** was wrong – which is why "the model is misbehaving" is so often really "we measured the wrong thing".

Before you reach for any of this

Every other chapter in this course ends with when the method fails. This one has to end with something blunter: **reinforcement learning is a last resort.**

Ask first	**If yes**
Can you obtain labelled examples of the right answer?	Do supervised learning. It is cheaper, more stable, and easier to debug.
Is there a single decision, not a sequence?	It is not an RL problem. Use a classifier or a bandit.
Do you know the dynamics and is the space small?	Plan directly – value iteration, search, or plain optimisation. No learning needed.

RL earns its cost only when **all** of these hold: the decisions are *sequential*, the consequences are *delayed*, nobody can tell you the right action – but you *can* score the outcome.

The price nobody puts on the slide

It is astonishingly data-hungry

DQN trained for **50 million frames** per game – about **38 days** of continuous play, for *one* Atari game. A human is competent in minutes.

Exploration costs in the real world

ϵ-greedy means *deliberately acting at random*. Free in a simulator. On a robot, a live recommender or a pricing system: broken hardware, lost users, lost money.

This is why almost all published RL successes are in domains with a **cheap, fast, resettable simulator** – games, and physics you can simulate – or, in the LLM case, a reward you can **check automatically** (does the code compile, does the answer match).

When you read that RL solved something, the first question is not "which algorithm?" It is **"where did the millions of attempts come from, and what did each cost?"**

One more thing, and it stings

Written by Rich Sutton – who also wrote the RL textbook.

The bitter lesson (Sutton, 2019)

> "The biggest lesson that can be read from 70 years of AI research is that **general methods that leverage computation are ultimately the most effective**, and by a large margin."

The pattern he documents, over and over: researchers encode human knowledge about the problem; it helps at first; then it is beaten by a simpler method that just *searches* or *learns* more, once the hardware allows.

Domain	The knowledge-engineered approach	What beat it
Chess	hand-crafted positional heuristics	massive search (Deep Blue)
Go	shape and pattern libraries	search + self-play RL (AlphaGo)
Speech	phonetic models, linguistic rules	statistical, then deep, learning
Vision	hand-designed feature detectors	learned features (CNNs)

It is *bitter* because it says the satisfying part – our insight into the problem – is usually not what wins. Sutton's point is that we keep re-learning this and keep resisting it.

Where this lecture is Exhibit A – and where it is not

Evidence for

- **AlphaGo → AlphaGo Zero.** Deleting the human game database made it *stronger*. Sutton's own example.
- **DQN.** One architecture, one set of hyperparameters, 49 Atari games.
- **DeepSeek-R1-Zero.** Nobody taught it to reason; the behaviour emerged from pure RL against a checkable reward. (The *shipped* R1 adds cold-start SFT first – same caveat as AlphaGo above.)

Read it carefully, though

- It is *not* "structure never helps". Convolution and attention are human-designed inductive biases, and both were decisive.
- Every method here is scaffolding: replay, target networks, clipping, KL penalties. Someone designed those.
- Compute is not free. "Wait for better hardware" is a strategy open to a few labs and nobody else.

The honest reading: prefer methods that **improve when given more compute and data**, and be suspicious of cleverness that does not. A claim about what to bet on – not a licence to stop thinking.

Recap

- RL is for problems with **no correct answer to imitate** – only delayed, sparse, non-differentiable consequences. The i.i.d. assumption is gone.
- An **MDP** $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ is the whole setting, and γ is a real modelling choice: it changed which reward our corridor agent walked towards.
- The **Bellman equation** is the one recursion everything solves or approximates. Solve it exactly with a model; **Q-learning** approximates it from experience without one. Tabular hits a wall, so Q becomes a **network** (DQN), with replay and a frozen target.
- **Policy gradients** optimise behaviour directly – necessary when the action space is a vocabulary. They are noisy: a baseline cut our wrong-direction rate from 33.9% to 0%. **PPO** then stops any update going too far.
- Optimising a *learned* reward invites **reward hacking**. The optimiser is not the problem; the objective is.
- **The bitter lesson:** bet on methods that improve with more compute and data. AlphaGo Zero got stronger by *deleting* the human games.

Next: the *LLM Training & Alignment* chapter – this lecture at scale. You can now read [10] InstructGPT (the pipeline above), [01] GRPO (drop the critic), [02] DPO (drop the reward model *and* the RL), and [11] DeepSeek-R1 (where reasoning emerged from pure RL in the R1-Zero variant).